

Resolución del Práctico 1

Testing de Software 2024

Índice

Introducción	3
1. Ejercicio 1	3
2. Ejercicio 2	3
2.1. Diferencia entre <i>fault</i> y <i>failure</i>	3
3. Ejercicio 3	3
3.1. Programa 1: <code>findLast(int[] x, int y)</code>	3
3.2. Programa 2: <code>lastZero(int[] x)</code>	4
3.3. Programa 3: <code>countPositive(int[] x)</code>	5
3.4. Programa 4: <code>oddOrPos(int[] x)</code>	5
3.5. Síntesis del ejercicio	6
4. Ejercicio 4	6
4.1. Reparaciones implementadas	6
4.2. Casos de prueba JUnit	6
4.3. Detección de falla usando el modelo RIPR	6
4.4. Ejecución de pruebas	7
5. Ejercicio 5	7
5.1. Análisis de <code>Point</code>	8
5.2. Análisis de <code>ColorPoint</code>	8
Ejercicio 6	9
6.1. Aclaración sobre el material provisto	9
6.2. a) Nuevos tests, falla detectada y corrección	9
6.3. b) Identificación de Arrange–Act–Assert	9

6.4. Ejecución de pruebas	10
7. Ejercicio 7	10
7.1. Aclaración sobre el material provisto	10
7.2. Tests implementados	10
7.3. Resultado y análisis	10
7.4. Identificación de Arrange–Act–Assert	10
Conclusiones	10

Introducción

El presente documento unifica la resolución del *Práctico 1* de la materia *Testing de Software 2024*. Se preserva el contenido técnico original de cada ejercicio y se mejora su presentación mediante una redacción académica, una estructura homogénea y una notación consistente.

1. Ejercicio 1

Lectura de los capítulos 1 y 2 del libro *Introduction to Software Testing*.

2. Ejercicio 2

2.1. Diferencia entre *fault* y *failure*

La diferencia central es la siguiente:

- **Fault (defecto):** problema interno en el software (código, diseño o especificación).
- **Failure (falla):** comportamiento externo incorrecto, observable durante la ejecución.

Además, entre ambos conceptos aparece **error**, entendido como un estado interno incorrecto que surge al ejecutar un defecto.

En términos causales:

$$fault \rightarrow error \rightarrow failure$$

Por lo tanto, un defecto puede existir incluso sin generar una falla visible en toda ejecución: para que la falla ocurra, el estado erróneo debe propagarse hasta una salida observable.

3. Ejercicio 3

Se analizaron cuatro programas defectuosos. Para cada caso se identifica el defecto, una corrección posible y los casos pedidos en los incisos b), c) y d).

3.1. Programa 1: `findLast(int[] x, int y)`

a) Defecto y corrección

El defecto está en la condición del ciclo:

```
1 for (int i = x.length - 1; i > 0; i--)
```

De esta forma nunca se evalúa el índice 0. La corrección es:

```
1 for (int i = x.length - 1; i >= 0; i--)
```

b) Caso que no ejecute el defecto

No es posible con entradas válidas: la condición defectuosa del `for` se evalúa en toda invocación del método.

c) Caso que ejecute el defecto y no produzca falla

`x = [5,2,3]`, `y = 2` Resultado esperado: 1.

d) Caso que ejecute el defecto y produzca falla

`x = [2,3,5]`, `y = 2` Esperado: 0. Resultado defectuoso: -1.

3.2. Programa 2: `lastZero(int[] x)`

a) Defecto y corrección

El método debe devolver el *último* índice con valor cero, pero recorre de izquierda a derecha y retorna el primer cero encontrado. La corrección consiste en recorrer desde el final.

```
1 for (int i = x.length - 1; i >= 0; i--) {  
2     if (x[i] == 0) {  
3         return i;  
4     }  
5 }
```

b) Caso que no ejecute el defecto

No es posible para arreglos no nulos, ya que la estrategia defectuosa forma parte del recorrido principal.

c) Caso que ejecute el defecto y no produzca falla

`x = [1,0,2]` Resultado esperado: 1.

d) Caso que ejecute el defecto y produzca falla

`x = [0,1,0]` Esperado: 2. Resultado defectuoso: 0.

3.3. Programa 3: countPositive(int[] x)

a) Defecto y corrección

Se cuenta $x[i] \geq 0$, lo que incluye al cero como positivo. La condición correcta es:

```
1 if (x[i] > 0) {  
2     count++;  
3 }
```

b) Caso que no ejecute el defecto

$x = []$. El cuerpo del for no se ejecuta.

c) Caso que ejecute el defecto y no produzca falla

$x = [-4, 2, 2]$ Resultado esperado: 2.

d) Caso que ejecute el defecto y produzca falla

$x = [-4, 2, 0, 2]$ Esperado: 2. Resultado defectuoso: 3.

3.4. Programa 4: oddOrPos(int[] x)

a) Defecto y corrección

La implementación defectuosa usa $x[i] \% 2 == 1$. En Java, los impares negativos no satisfacen esa condición (por ejemplo, $-3 \% 2 == -1$).

La corrección es:

```
1 if (x[i] % 2 != 0 || x[i] > 0) {  
2     count++;  
3 }
```

b) Caso que no ejecute el defecto

$x = []$. El cuerpo del for no se ejecuta.

c) Caso que ejecute el defecto y no produzca falla

$x = [2, 4, -2]$ Resultado esperado: 2.

d) Caso que ejecute el defecto y produzca falla

$x = [-3, -2, 0, 1, 4]$ Esperado: 3. Resultado defectuoso: 2.

3.5. Síntesis del ejercicio

- `findLast`: no evalúa el índice 0.
- `lastZero`: devuelve el primer cero en vez del último.
- `countPositive`: contabiliza el 0 como positivo.
- `oddOrPos`: no reconoce impares negativos.

4. Ejercicio 4

Se implementaron las reparaciones del ejercicio anterior y se incorporaron casos de prueba JUnit para verificar el comportamiento corregido.

4.1. Reparaciones implementadas

- `findLast`: cambio de condición de recorrido de `i > 0` a `i >= 0`.
- `lastZero`: recorrido desde el final del arreglo.
- `countPositive`: reemplazo de `>= 0` por `> 0`.
- `oddOrPos`: reemplazo de `% 2 == 1` por `% 2 != 0`.

4.2. Casos de prueba JUnit

Se implementaron los mismos casos del enunciado que evidenciaban fallas en las versiones defectuosas:

- `findLast([2,3,5], 2) = 0`
- `lastZero([0,1,0]) = 2`
- `countPositive([-4,2,0,2]) = 2`
- `oddOrPos([-3,-2,0,1,4]) = 3`

4.3. Detección de falla usando el modelo RIPR

1) `findLast`

- **Reachability**: el test alcanza la condición defectuosa del `for`.
- **Infection**: se omite la revisión de `x[0]`.
- **Propagation**: se retorna un índice incorrecto.

- **Revealability:** la aserción muestra diferencia entre esperado y obtenido.

2) lastZero

- **Reachability:** se ejecuta el recorrido de izquierda a derecha.
- **Infection:** se selecciona un candidato incorrecto (primer cero).
- **Propagation:** ese índice se retorna como resultado final.
- **Revealability:** la aserción detecta 0 en lugar de 2.

3) countPositive

- **Reachability:** se evalúa $x[i] \geq 0$.
- **Infection:** el contador se incrementa indebidamente para 0.
- **Propagation:** el total final queda sobreestimado.
- **Revealability:** la aserción muestra 3 en lugar de 2.

4) oddOrPos

- **Reachability:** se ejecuta la condición lógica del método.
- **Infection:** los impares negativos no se contabilizan.
- **Propagation:** el conteo final queda una unidad por debajo.
- **Revealability:** la aserción detecta 2 en lugar de 3.

4.4. Ejecución de pruebas

Desde la carpeta del ejercicio:

```
mvn -Dmaven.repo.local=.m2 test
```

5. Ejercicio 5

Se analizaron las clases `Point` y `ColorPoint` del paquete `practico1_exercises.color_points`, en el contexto del problema clásico de `equals` en jerarquías por herencia.

5.1. Análisis de Point

a) Problema y modificación propuesta

`Point` define igualdad por valor y permite que subclases también sean tratadas como `Point` (uso de `instanceof Point`). Esto habilita inconsistencias cuando una subclase agrega estado adicional (por ejemplo, `color`).

La solución recomendada es evitar extender `Point` para agregar estado de valor: usar composición (o, alternativamente, declarar `Point` como `final`).

b) Test que no ejecute la falla

Comparación entre dos instancias de `Point` con mismas coordenadas.

c) Test que ejecute la falla sin error observable

Comparación `Point(1,2)` con `ColorPoint(2,3,RED)`. Aunque se ejecuta la lógica problemática de comparación entre tipos, el resultado sigue siendo correcto (`false`) por diferencia de coordenadas.

d) Test con error pero sin falla

No se considera posible en este caso: cuando aparece el error lógico en `equals`, se manifiesta directamente como comportamiento observable incorrecto.

5.2. Análisis de ColorPoint

a) Problema y modificación propuesta

`ColorPoint` redefine `equals` exigiendo `instanceof ColorPoint`, lo que rompe simetría con `Point`. Por ejemplo, puede ocurrir:

```
p.equals(cp) == true y cp.equals(p) == false
```

La corrección aplicada consiste en usar composición en lugar de herencia (implementada en `ColorPointFixed`).

b) Test que no ejecute la falla

Comparación entre dos `ColorPoint` con mismas coordenadas y color.

c) Test que ejecute la falla sin error observable

Comparación `ColorPoint(1,2,RED)` con `Point(4,5)`. Se ejecuta la rama conflictiva, pero el resultado esperado sigue siendo `false`.

d) Test con error pero sin falla

No se considera posible, por la misma razón conceptual indicada para `Point`.

Ejercicio 6

6.1. Aclaración sobre el material provisto

En el material disponible no estaban provistas la clase `PointTest` ni el paquete `practico1_exercises`. Por ello se crearon:

- una implementación de `Point`,
- una clase de pruebas `PointTest`,
- tests adicionales para cubrir comportamiento en `HashSet`.

6.2. a) Nuevos tests, falla detectada y corrección

Se incorporaron pruebas para:

- igualdad entre puntos equivalentes,
- desigualdad entre puntos distintos,
- tratamiento de duplicados en `HashSet`,
- búsqueda de elementos equivalentes en `HashSet`.

La falla detectada fue la inconsistencia entre `equals` y `hashCode`. La corrección aplicada fue:

```
1 @Override
2 public int hashCode() {
3     return Objects.hash(x, y);
4 }
```

6.3. b) Identificación de Arrange–Act–Assert

En cada test se dejaron explícitas las tres fases:

- **Arrange:** preparación del escenario y datos.
- **Act:** ejecución de la operación bajo prueba.
- **Assert:** verificación de resultados esperados.

6.4. Ejecución de pruebas

```
mvn -Dmaven.repo.local=.m2 test
```

7. Ejercicio 7

7.1. Aclaración sobre el material provisto

No se encontraron implementaciones dadas de `PointSet` ni `PointSetTest`. Para resolver el ejercicio se crearon ambas clases y se diseñaron pruebas adicionales.

7.2. Tests implementados

Se incorporaron casos para verificar:

- estado inicial vacío,
- no inserción de duplicados equivalentes,
- reconocimiento de equivalencia en `contains`,
- eliminación de equivalentes en `remove`,
- respuesta negativa de `contains` ante ausencias reales.

7.3. Resultado y análisis

Los tests pasan sobre la implementación actual. No se detectaron fallas adicionales en `PointSet`. Este resultado es consistente con la corrección previa de `Point` (coherencia entre `equals` y `hashCode`), dado que `PointSet` se apoya en un `HashSet<Point>`.

7.4. Identificación de Arrange–Act–Assert

Al igual que en el ejercicio 6, todos los tests se estructuraron con fases explícitas de *arrange*, *act* y *assert*.

Conclusiones

La resolución del Práctico 1 permitió:

- distinguir formalmente los conceptos de *fault*, *error* y *failure*;
- analizar defectos concretos en programas imperativos y su reparación;
- aplicar el modelo RIPR para explicar detección de fallas;

- abordar un problema clásico de diseño orientado a objetos en el contrato de `equals`;
- reforzar diseño de pruebas unitarias y consistencia semántica en estructuras de datos basadas en hashing.